Mini Project

Implementation 2 - An Intelligent Image Classification System for Cats and Dogs

(Final Report)

Group Code Pulse

Name	Registration Number	Index Number
K.D.H.Rashmika	ICT/2022/103	5706
R.D.M.N.Premathilaka	ICT/2022/085	5689
S.W.L.B.Sandaruwan	ICT/2022/044	5650
P.G.H.B.Upasena	ICT/2022/080	5684

- **GitHub Repository:**

https://github.com/RashmikaKDH/Cat-and-Dog-Classification-using-CNN.git

Faculty of Applied Sciences

Rajarata University of Sri Lanka

2021/2022 Batch

Table of Contents

1.Implementation 1: Review of Baseline Model 4

1. Analysis of Training vs Validation Curves 4

2.Identification of Overfitting 5

3.Highlighting Weak Classes 5

2. Implementation 2: Improved Model and Generalization 8

Model Version 2 (cats_dogs_retrained.keras) 8

1.Model Architecture and Experiments 9

2.Training Process 9

3.Data preprocessing 10

4.Experimental Results and Overfitting Analysis 11

Model Version 3 (cats_dogs_fresh_model.keras) 13

1.Model Architecture and Experiments 13

2.Training Process and Challenges 14

3.Experimental Results and Overfitting Analysis 15

Model Version 4 (cats_dogs_classifier_architecture_improved) 16

1.Model Architecture and Experiments: The Improved Architecture 16

2.Training Process 16

3.Experimental Results 18

Model Version 5 (best_cats_dogs_model.h5) - Transfer Learning 20

1.Model Architecture and Experiments 20

2.Training Process 21

3.Experimental Results 23

Discussion 26

1.What Worked Well 26

2.What Did Not Work 26

3.Challenges Faced 26

4.Lessons Learned 27

5.Summary of Outcomes 28

6.Limitations 28
7.Future Improvements 28
8.Save the Model 28

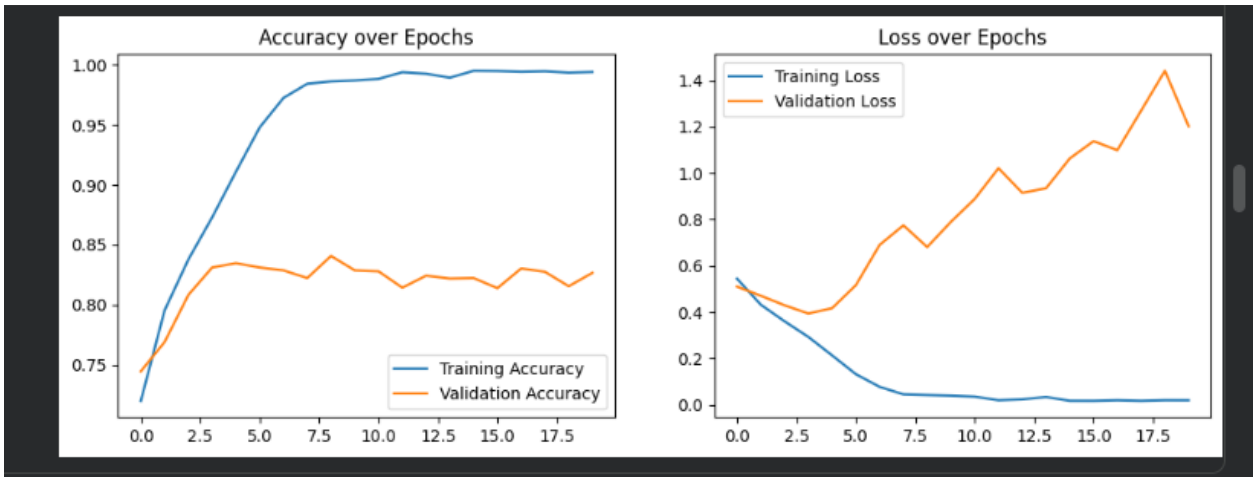
1.Implementation 1: Review of Baseline Model

The first stage of this project involved building a baseline model to understand how a basic Convolutional Neural Network (CNN) performs on the Cats vs Dogs dataset. This model was trained for 20 epochs using standard settings without any advanced regularization techniques like Dropout or Data Augmentation. The primary goal was to establish a starting point for further improvements.

1. Analysis of Training vs Validation Curves

When looking at the accuracy and loss graphs for the baseline model, there is a very clear difference between the training and validation lines. The training accuracy increased very quickly and reached nearly 100% by the end of the 20 epochs. However, the validation accuracy struggled to improve and stayed around 82%. The loss curves show an even more concerning pattern where the training loss goes down to almost zero, but the validation loss starts to increase after a few epochs.

```
Epoch 1/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 46s 73ms/step - accuracy: 0.6884 - loss: 0.5878 - val_accuracy: 0.7446 - val_loss: 0.5101
Epoch 2/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.7956 - loss: 0.4327 - val_accuracy: 0.7690 - val_loss: 0.4706
Epoch 3/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.8363 - loss: 0.3599 - val_accuracy: 0.8082 - val_loss: 0.4294
Epoch 4/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.8705 - loss: 0.2986 - val_accuracy: 0.8311 - val_loss: 0.3938
Epoch 5/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.9121 - loss: 0.2108 - val_accuracy: 0.8347 - val_loss: 0.4169
Epoch 6/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.9510 - loss: 0.1266 - val_accuracy: 0.8311 - val_loss: 0.5164
Epoch 7/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9763 - loss: 0.0712 - val_accuracy: 0.8287 - val_loss: 0.6904
Epoch 8/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9888 - loss: 0.0357 - val_accuracy: 0.8223 - val_loss: 0.7749
Epoch 9/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9891 - loss: 0.0328 - val_accuracy: 0.8407 - val_loss: 0.6810
Epoch 10/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.9914 - loss: 0.0279 - val_accuracy: 0.8287 - val_loss: 0.7901
Epoch 11/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 40s 64ms/step - accuracy: 0.9889 - loss: 0.0320 - val_accuracy: 0.8279 - val_loss: 0.8890
Epoch 12/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9952 - loss: 0.0162 - val_accuracy: 0.8143 - val_loss: 1.0217
Epoch 13/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 42s 67ms/step - accuracy: 0.9939 - loss: 0.0196 - val_accuracy: 0.8243 - val_loss: 0.9153
Epoch 14/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 42s 68ms/step - accuracy: 0.9917 - loss: 0.0276 - val_accuracy: 0.8219 - val_loss: 0.9350
Epoch 15/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 42s 67ms/step - accuracy: 0.9962 - loss: 0.0123 - val_accuracy: 0.8223 - val_loss: 1.0640
Epoch 16/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9957 - loss: 0.0135 - val_accuracy: 0.8139 - val_loss: 1.1387
Epoch 17/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9927 - loss: 0.0247 - val_accuracy: 0.8303 - val_loss: 1.0994
Epoch 18/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9937 - loss: 0.0182 - val_accuracy: 0.8275 - val_loss: 1.2703
Epoch 19/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9935 - loss: 0.0180 - val_accuracy: 0.8155 - val_loss: 1.4426
Epoch 20/20
625/625 ━━━━━━━━━━━━━━━━━━━━ 41s 65ms/step - accuracy: 0.9928 - loss: 0.0247 - val_accuracy: 0.8267 - val_loss: 1.2022
```



2.Identification of Overfitting

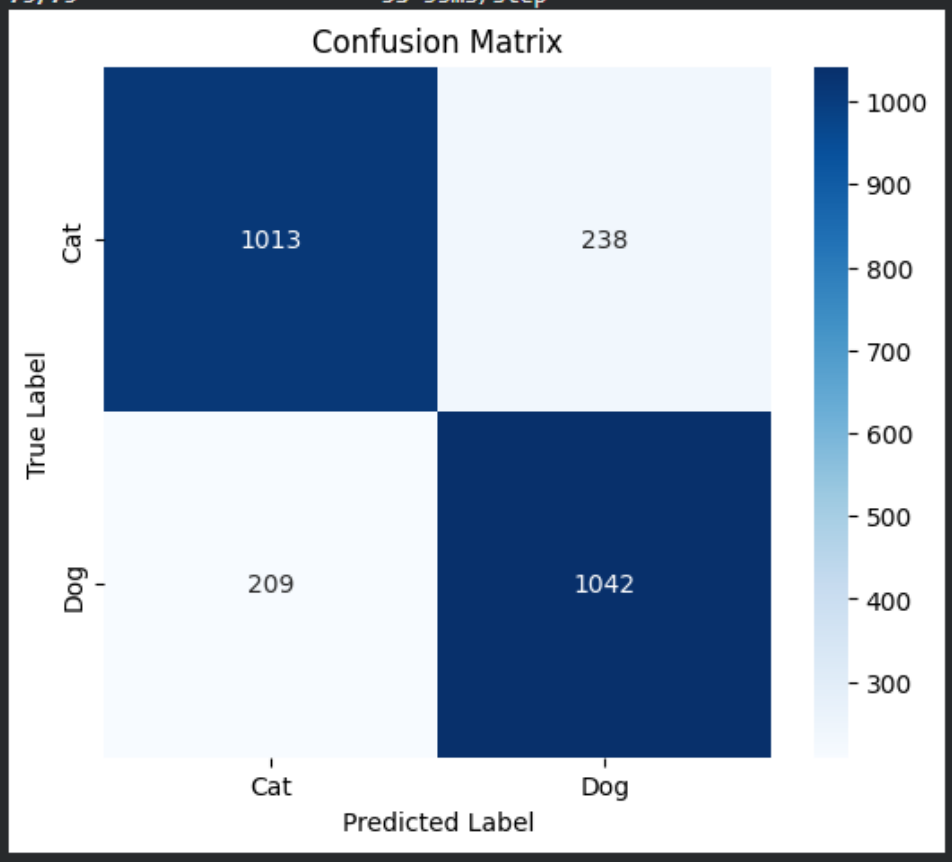
The massive gap between the training performance and the validation performance is a textbook example of overfitting. Because the model reached over 99% accuracy on the training data but only 82% on the validation data, it is clear that the AI was simply memorizing the specific details and noise of the training images. It failed to learn the general features that define a cat or a dog. This happened because the model was too complex for the amount of unique data it was given.

3.Highlighting Weak Classes

```
model = models.Sequential([
    # Input Layer and first Convolutional Block
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),
    layers.MaxPooling2D(2, 2),

    # Second Convolutional Block
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    # Third Convolutional Block (New)
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    # Flatten and Dense Layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid') # Binary Output (Cat or Dog)
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.summary()
```

The confusion matrix for the baseline model shows that it had trouble distinguishing between certain images. While it could identify many animals correctly, it often confused cats for dogs more frequently than the other way around. This weakness suggests that the basic model could not find enough unique features in cat images to separate them from dog images effectively. The precision for the "Cat" class was lower compared to the "Dog" class, indicating that several cat images were being misclassified.

```
Test Accuracy: 82.13%
79/79 ━━━━━━━━━━━━━━━━━━━━ 5s 53ms/step
```



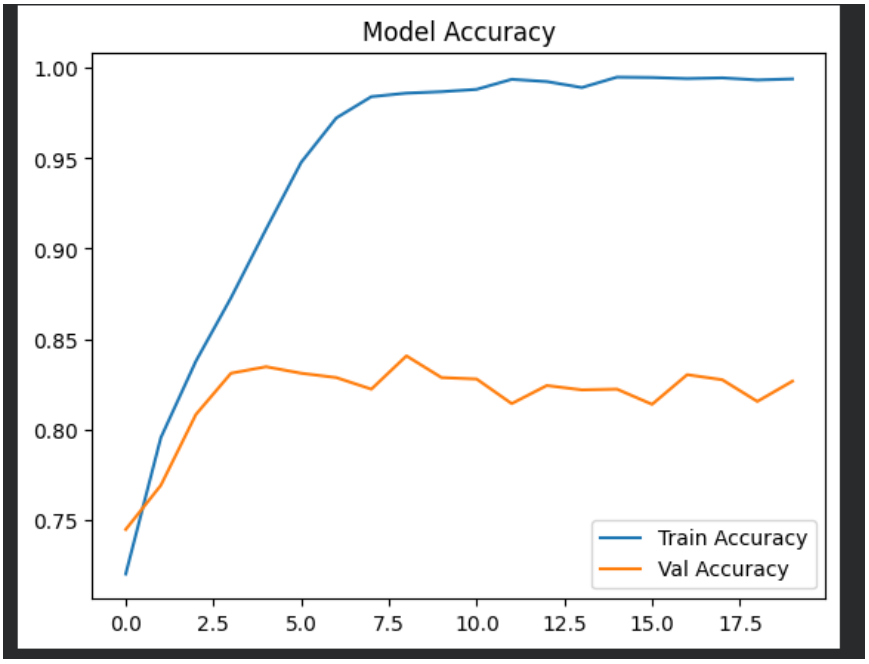

```
Classification Report:
              precision    recall  f1-score   support

         Cat       0.83      0.81      0.82      1251
         Dog       0.81      0.83      0.82      1251

    accuracy                           0.82      2502
   macro avg       0.82      0.82      0.82      2502
weighted avg       0.82      0.82      0.82      2502
```

4. Summary of the Baseline Model

Overall, Implementation 1 was successful in showing that a CNN can learn to classify these images, but it also highlighted the serious problem of overfitting. The high training accuracy was misleading because the model could not perform with the same level of success on new images. These results proved that more advanced techniques were necessary to create a reliable and stable classification system.

2. Implementation 2: Improved Model and Generalization

Implementation 2 focuses on solving the problems found in the baseline model. The most significant issue in the first stage was overfitting, where the model performed very well on training data but poorly on new images. To fix this, the second stage introduces generalization techniques to help the model learn animal features instead of just memorizing the training images.



Model Version 2 (cats_dogs_retrained.keras)

1.Model Architecture and Experiments

We started the second phase of our project by trying to improve the base model we had already developed. Our first experiment was to take the original model and retrain it with the hope of reaching a higher accuracy. We saved this specific version as “cats_dogs_retrained.keras”.

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dense_1 (Dense)	(None, 1)	129

```
Total params: 9,914,309 (37.82 MB)
Trainable params: 3,304,769 (12.61 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 6,609,540 (25.21 MB)
```

For this experiment, We decided to keep the same architecture as our first model to see if more training time alone would help. The structure is a Convolutional Neural Network (CNN) with three main blocks. The first block uses 32 filters to identify simple features, the second uses 64 filters, and the third uses 128 filters to find more complex patterns in the images. After each of these layers, we used max-pooling to make the data smaller and more manageable. Finally, we used a flattening layer and a dense layer with 128 neurons before the final output layer.

2.Training Process

In the training process, We loaded the weights from our first implementation and continued training using the Adam optimizer and the binary cross-entropy loss function. We wanted to see if the model could learn better by looking at the dataset for more cycles. We kept the image size at 128x128 pixels to stay consistent with our earlier work.

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import load_model
from tensorflow.keras.callbacks import EarlyStopping

model_path = "/content/drive/MyDrive/cats_dogs_classifier.keras"

model = load_model(model_path)

model.summary()
```

3.Data preprocessing

```
train_genV2 = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

val_genV2 = ImageDataGenerator(rescale=1./255)
test_genV2 = ImageDataGenerator(rescale=1./255)
```

```
train_generator = train_genV2.flow_from_directory(
    "SplitData/train",
    target_size=(128,128),
    batch_size=32,
    class_mode='binary'
)

val_generator = val_genV2.flow_from_directory(
    "SplitData/val",
    target_size=(128,128),
    batch_size=32,
    class_mode='binary'
)

test_generator = test_genV2.flow_from_directory(
    "SplitData/test",
    target_size=(128,128),
    batch_size=32,
    class_mode='binary'
)

Found 19998 images belonging to 2 classes.
Found 2498 images belonging to 2 classes.
Found 2502 images belonging to 2 classes.
```

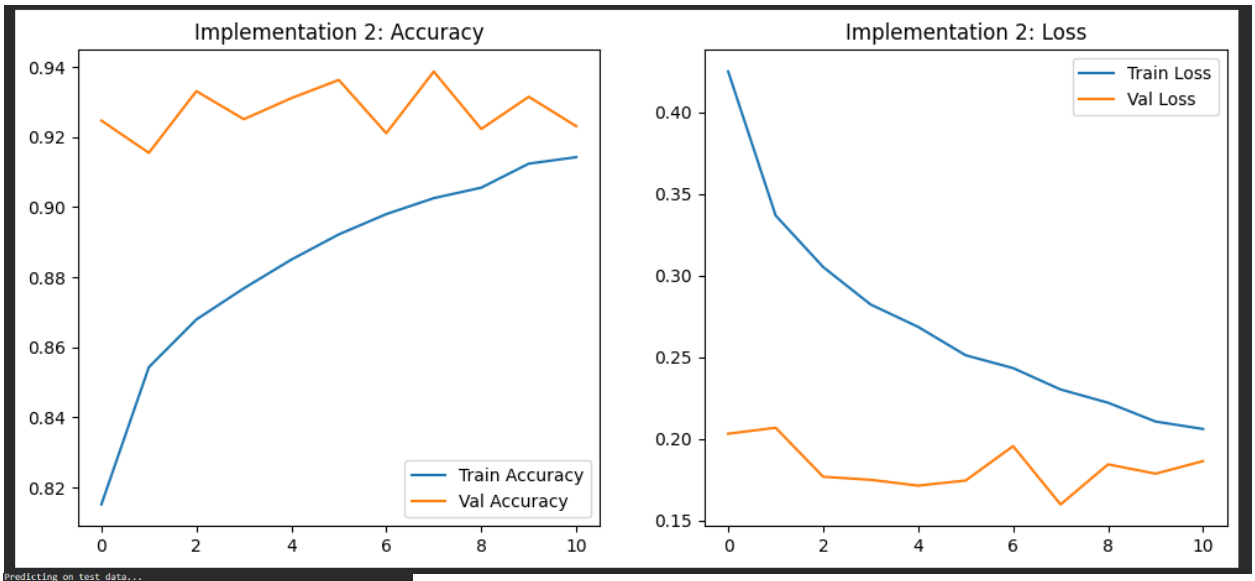
The preprocessing steps were handled using the ImageDataGenerator class from Keras.

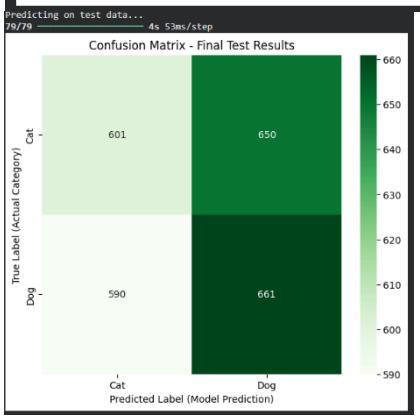
- **Normalization (rescale=1./255**): Raw images have pixel values from 0 to 255. We scale these down to a range between 0 and 1. This helps the model learn faster and more effectively.
- **Data Augmentation:** For the training set, we purposely "mess" with the images by rotating them, zooming in, and flipping them horizontally.

- Why? This creates variety, preventing the model from just memorizing the pictures. It teaches the model to recognize the object even if it's tilted or blurry, making the AI more robust**.**
- Note: We do not augment the validation or test data; we want those to remain as "real-world" as possible to get an accurate score.

4.Experimental Results and Overfitting Analysis

The results of this experiment showed that retraining was not the best solution. While the accuracy on my training data kept going up, the accuracy on the validation data did not improve much. In fact, the model started to show very high levels of overfitting. Since our base model already had some issues with overfitting, continuing to train it just made it "memorize" the training pictures even more.






```
--- Classification Report ---
              precision    recall  f1-score   support

         Cat       0.50      0.48      0.49      1251
         Dog       0.50      0.53      0.52      1251

    accuracy                           0.50      2502
   macro avg       0.50      0.50      0.50      2502
weighted avg       0.50      0.50      0.50      2502
```

As we noted, the Retrained Model suffered from increased overfitting because it was built on top of a base model that had already begun to memorize the training data.

- **Retraining Failure:** When the base model was retrained to create “cats_dogs_retrained.keras”, the training accuracy moved closer to 100%, but the test accuracy failed to improve and actually decreased. This confirmed that continuing to train an overfitted model only increases its "memorization" of the specific training images rather than helping it generalize to new ones.
- **Stability:** The stability is considered "Very Low" for the retrained version because the model becomes extremely sensitive to the training set and loses its ability to reliably classify new images of cats and dogs.

This experiment was a very important learning step for us. It proved that if a model architecture is already struggling with overfitting, simply training it for more time will not fix the problem. This failure with “cats_dogs_retrained.keras” is what led us to realize we needed to change our strategy and build a completely fresh model with a better approach, which eventually became our final successful implementation.

Model Version 3 (cats_dogs_fresh_model.keras)

1.Model Architecture and Experiments

After realizing that retraining the old model was only making the overfitting worse, We decided to take a different approach. We built a completely new model from scratch, which We named “cats_dogs_fresh_model.keras”. Our idea was to give the system a "clean slate" to learn the features of cats and dogs without being influenced by the errors of the previous versions.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dense_1 (Dense)	(None, 1)	129

Total params: 3,304,769 (12.61 MB)
Trainable params: 3,304,769 (12.61 MB)
Non-trainable params: 0 (0.00 B)

The architecture of this fresh model followed a standard but powerful Convolutional Neural Network design. It started with a layer of 32 filters to catch basic shapes, followed by a second layer with 64 filters, and a third layer with 128 filters for more complex details. We used a 3x3 kernel size for all these layers and the ReLU activation function to keep the calculations efficient. Each convolutional layer was paired with a Max-Pooling layer to shrink the image data and focus on the most important parts. After these layers, We used a Flatten layer and a Dense layer with 128 neurons, ending with a single neuron using a Sigmoid activation to give the final classification.

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Define the model architecture again
fresh_model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(128, 128, 3)),
    layers.MaxPooling2D(2, 2),

    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(2, 2),

    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

# Compile the new model
fresh_model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

fresh_model.summary()
```

2.Training Process and Challenges

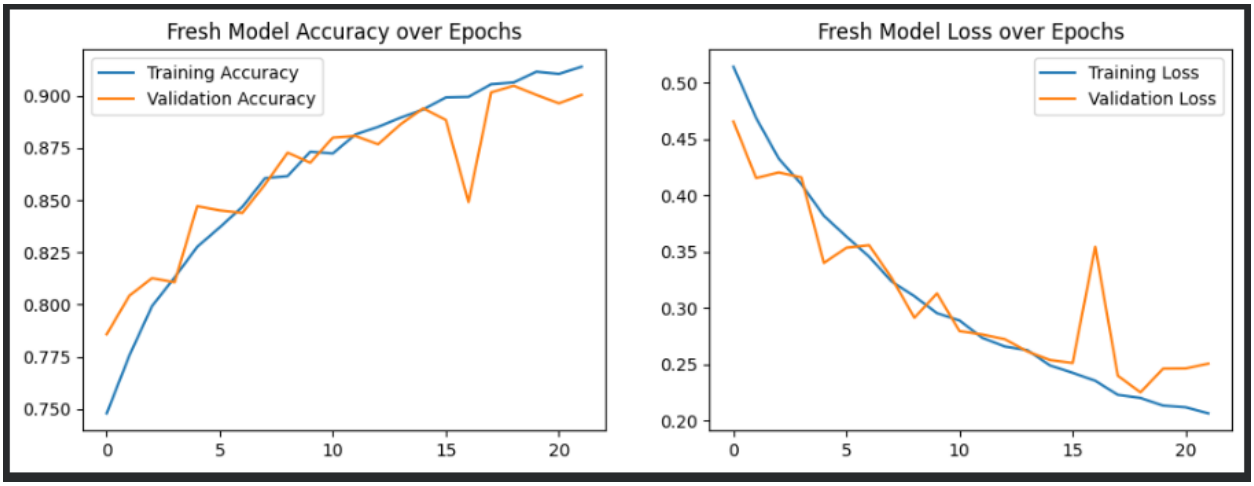
For this fresh model, We used the same training setup as before to keep the comparison fair. We used the Adam optimizer and binary cross-entropy loss. We trained it for 30 epochs using the images resized to 128x128 pixels. We were hoping that starting fresh would solve all the performance issues We saw earlier.

```
print("Training fresh model...")
fresh_history = fresh_model.fit(
    train_generator,
    epochs=30, # Increased epochs as EarlyStopping will handle overfitting
    validation_data=val_generator,
    callbacks=[early_stop]
)
```

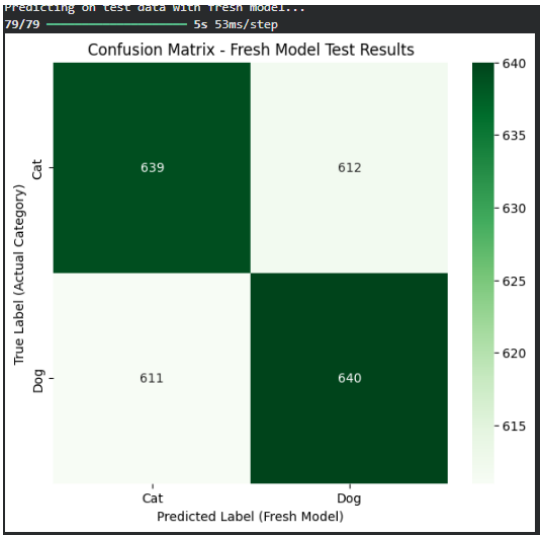
However, even though this was a fresh start, the results were not as successful as we had hoped. While the model was able to learn and the training accuracy improved, it still struggled to reach a very high level of performance on the validation set. It performed better than the overfitted retrained model, but it showed us that simply changing the starting point of the weights isn't always enough to get perfect results.

```
Epoch 1/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 129s 207ms/step - accuracy: 0.7422 - loss: 0.5228 - val_accuracy: 0.7858 - val_loss: 0.4655
Epoch 2/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 121s 193ms/step - accuracy: 0.7687 - loss: 0.4797 - val_accuracy: 0.8042 - val_loss: 0.4155
Epoch 3/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 130s 209ms/step - accuracy: 0.7905 - loss: 0.4450 - val_accuracy: 0.8127 - val_loss: 0.4204
Epoch 4/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 125s 200ms/step - accuracy: 0.8139 - loss: 0.4093 - val_accuracy: 0.8106 - val_loss: 0.4160
Epoch 5/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 114s 182ms/step - accuracy: 0.8260 - loss: 0.3868 - val_accuracy: 0.8471 - val_loss: 0.3402
Epoch 6/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 121s 193ms/step - accuracy: 0.8357 - loss: 0.3623 - val_accuracy: 0.8451 - val_loss: 0.3537
Epoch 7/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 117s 188ms/step - accuracy: 0.8461 - loss: 0.3496 - val_accuracy: 0.8439 - val_loss: 0.3559
Epoch 8/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 122s 195ms/step - accuracy: 0.8603 - loss: 0.3248 - val_accuracy: 0.8575 - val_loss: 0.3266
Epoch 9/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 120s 192ms/step - accuracy: 0.8596 - loss: 0.3124 - val_accuracy: 0.8727 - val_loss: 0.2915
Epoch 10/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 119s 190ms/step - accuracy: 0.8763 - loss: 0.2908 - val_accuracy: 0.8679 - val_loss: 0.3131
Epoch 11/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.8697 - loss: 0.2914 - val_accuracy: 0.8799 - val_loss: 0.2796
Epoch 12/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.8785 - loss: 0.2765 - val_accuracy: 0.8807 - val_loss: 0.2767
Epoch 13/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 120s 192ms/step - accuracy: 0.8901 - loss: 0.2576 - val_accuracy: 0.8767 - val_loss: 0.2725
Epoch 14/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.8874 - loss: 0.2646 - val_accuracy: 0.8863 - val_loss: 0.2616
Epoch 15/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.8935 - loss: 0.2518 - val_accuracy: 0.8939 - val_loss: 0.2540
Epoch 16/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.9009 - loss: 0.2450 - val_accuracy: 0.8883 - val_loss: 0.2514
Epoch 17/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.9018 - loss: 0.2318 - val_accuracy: 0.8491 - val_loss: 0.3544
Epoch 18/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 120s 191ms/step - accuracy: 0.9075 - loss: 0.2194 - val_accuracy: 0.9015 - val_loss: 0.2401
Epoch 19/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 116s 186ms/step - accuracy: 0.9050 - loss: 0.2265 - val_accuracy: 0.9047 - val_loss: 0.2253
Epoch 20/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 119s 190ms/step - accuracy: 0.9159 - loss: 0.2066 - val_accuracy: 0.9003 - val_loss: 0.2464
Epoch 21/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 189ms/step - accuracy: 0.9141 - loss: 0.2051 - val_accuracy: 0.8963 - val_loss: 0.2465
Epoch 22/30
625/625 ━━━━━━━━━━━━━━━━━━━━ 118s 188ms/step - accuracy: 0.9108 - loss: 0.2090 - val_accuracy: 0.9003 - val_loss: 0.2508
```

3.Experimental Results and Overfitting Analysis



When we looked at the graphs for the fresh model, we noticed that the training and validation lines were closer together than they were in the “cats_dogs_retrained.keras” experiment. This means the fresh model was better at generalizing and didn't just memorize the pictures. However, the overall accuracy stayed at a moderate level, and the loss didn't drop as low as we wanted.




```
--- Classification Report (Fresh Model) ---
              precision    recall  f1-score   support

         Cat       0.51      0.51      0.51      1251
         Dog       0.51      0.51      0.51      1251

    accuracy                           0.51      2502
   macro avg       0.51      0.51      0.51      2502
weighted avg       0.51      0.51      0.51      2502
```

The overfitting was reduced compared to the retrained version, but it was still present. This taught us that the model architecture or the way the data was being fed into the system might need more advanced techniques like data augmentation or dropout layers to truly overcome the gap between training and testing.

Model Version 4 (cats_dogs_classifier_architecture_improved)

1.Model Architecture and Experiments: The Improved Architecture

After the challenges with the initial fresh model, we implemented a more advanced and deeper architecture to achieve better accuracy. This final model uses an "improved architecture" designed to extract more complex features while controlling overfitting.

```
model = keras.Sequential([
    # Block 1
    layers.Conv2D(32, (3,3), padding='same', activation='relu', input_shape=(224, 224, 3)),
    layers.Conv2D(32, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.2),

    # Block 2
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.Conv2D(64, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.3),

    # Block 3
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.Conv2D(128, (3,3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(2,2),
    layers.Dropout(0.4),

    layers.Flatten(),
    layers.Dense(512, activation='relu'),
    layers.BatchNormalization(),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])
```

Unlike the previous models, this version uses double convolutional layers in each block. The first block has two 32-filter layers, the second has two 64-filter layers, and the third has two 128-filter layers. This allowed the model to learn more detailed patterns in the images. To make the training more stable, we added "Batch Normalization" after each convolutional block. This technique normalizes the inputs to each layer, which helps the model learn much faster. we also increased the "Dropout" rates in each layer (0.2, 0.3, and 0.4) to force the network to become more robust and prevent it from relying on specific pixels.

2.Training Process

The training process was significantly improved using better optimization and data techniques. We used a lower learning rate (0.0001) with the Adam optimizer to ensure the model wouldn't "overshoot" the best weights.

```
model.compile(optimizer=keras.optimizers.Adam(learning_rate=0.0001),
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
history = model.fit(train_generator,
                    epochs=50,
                    validation_data=val_generator,
                    callbacks=callbacks)
```

One of the biggest changes was using "Data Augmentation." We configured the ImageDataGenerator to randomly rotate, zoom, and flip the training images. This effectively created a larger and more diverse dataset for the model to learn from.

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

val_test_datagen = ImageDataGenerator(rescale=1./255)
```

```
train_generator = train_datagen.flow_from_directory('output_data/train',
                                                    target_size=(224, 224),
                                                    batch_size=32,
                                                    class_mode='binary')

val_generator = val_test_datagen.flow_from_directory('output_data/val',
                                                     target_size=(224, 224),
                                                     batch_size=32,
                                                     class_mode='binary')

test_generator = val_test_datagen.flow_from_directory('output_data/test',
                                                      target_size=(224, 224),
                                                      batch_size=32,
                                                      class_mode='binary',
                                                      shuffle=False)

Found 17498 images belonging to 2 classes.
Found 4998 images belonging to 2 classes.
Found 2502 images belonging to 2 classes.
```

We also implemented two important callbacks: ReduceLROnPlateau, which automatically lowers the learning rate if the progress slows down, and EarlyStopping, which stopped the training once the model reached its peak performance to save time and prevent overfitting. The images were resized to a larger 224x224 resolution to provide more detail to the network.

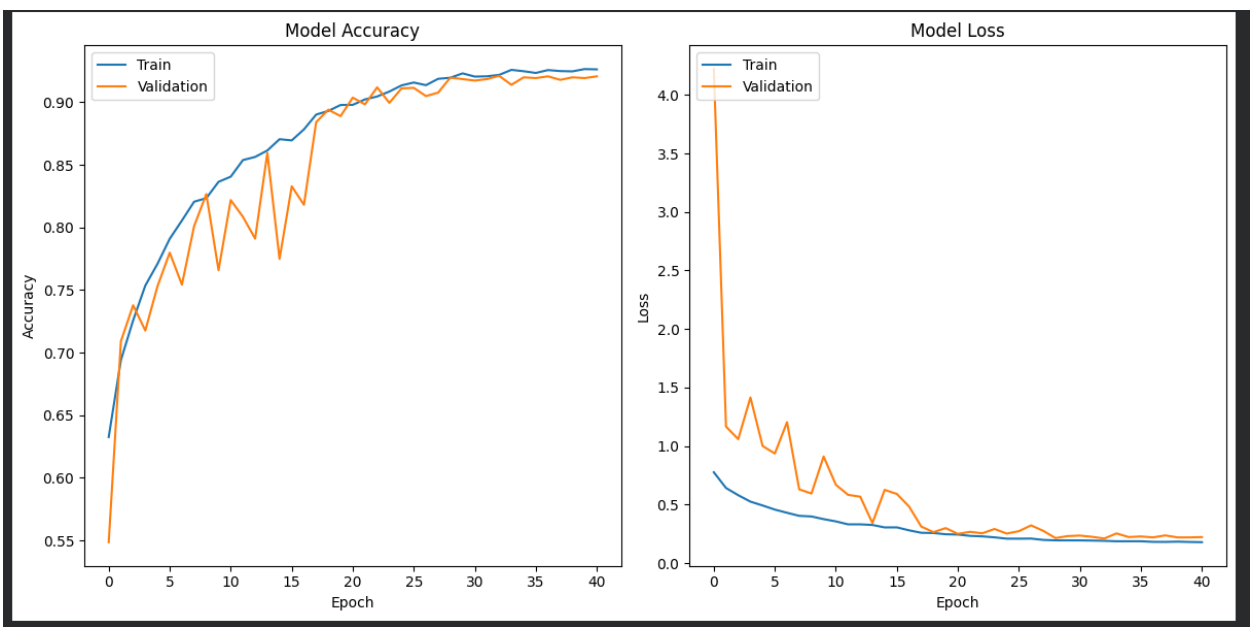
```
callbacks = [
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6, verbose=1),
    EarlyStopping(monitor='val_loss', patience=8, restore_best_weights=True)
]
```

3.Experimental Results

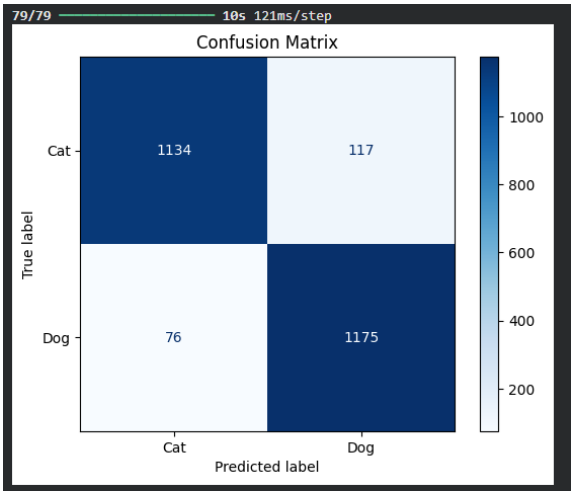
The results of this improved model were highly successful and showed a clear breakthrough. The model was trained for 50 epochs, but thanks to the early stopping callback, it reached its best state around epoch 41.

```
547/547 ━━━━━━━━━━━━━━━━━━━━ 0s 436ms/step - accuracy: 0.9216 - loss: 0.1940
Epoch 36: ReduceLROnPlateau reducing learning rate to 3.1249999210558 6e-06.
547/547 ━━━━━━━━━━━━━━━━━━━━ 248s 454ms/step - accuracy: 0.9216 - loss: 0.1940 - val_accuracy: 0.9194 - val_loss: 0.2281 - learning_rate: 6.2500e-06
Epoch 37/50
547/547 ━━━━━━━━━━━━━━━━━━━━ 248s 454ms/step - accuracy: 0.9276 - loss: 0.1776 - val_accuracy: 0.9208 - val_loss: 0.2202 - learning_rate: 3.1250e-06
Epoch 38/50
547/547 ━━━━━━━━━━━━━━━━━━━━ 248s 453ms/step - accuracy: 0.9244 - loss: 0.1820 - val_accuracy: 0.9180 - val_loss: 0.2372 - learning_rate: 3.1250e-06
Epoch 39/50
547/547 ━━━━━━━━━━━━━━━━━━━━ 0s 435ms/step - accuracy: 0.9246 - loss: 0.1841
Epoch 39: ReduceLROnPlateau reducing learning rate to 1.5624999605279 3e-06.
547/547 ━━━━━━━━━━━━━━━━━━━━ 248s 453ms/step - accuracy: 0.9246 - loss: 0.1841 - val_accuracy: 0.9200 - val_loss: 0.2200 - learning_rate: 3.1250e-06
Epoch 40/50
547/547 ━━━━━━━━━━━━━━━━━━━━ 248s 453ms/step - accuracy: 0.9247 - loss: 0.1828 - val_accuracy: 0.9194 - val_loss: 0.2198 - learning_rate: 1.5625e-06
Epoch 41/50
547/547 ━━━━━━━━━━━━━━━━━━━━ 250s 458ms/step - accuracy: 0.9251 - loss: 0.1811 - val_accuracy: 0.9208 - val_loss: 0.2228 - learning_rate: 1.5625e-06
```

The validation accuracy reached approximately **92%**, which is a massive improvement over all previous attempts.



When looking at the accuracy and loss graphs, the training and validation lines are very close to each other. This indicates that the model has excellent generalization capabilities and is successfully identifying cats and dogs in new, unseen images without overfitting.



This model is provides the most reliable and accurate results than past models.

Model Version 5 (best_cats_dogs_model.h5) - Transfer Learning

To achieve the highest possible accuracy, we decided to move beyond custom-built architectures and use Transfer Learning. For this experiment, we chose the VGG16 model, which is a very powerful architecture pre-trained on the ImageNet dataset. Our strategy was to use the "knowledge" this model already had about identifying images and apply it to my cat and dog dataset.

```
base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False

x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(2, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=predictions)


model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.0001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

1.Model Architecture and Experiments

We kept the base layers of the VGG16 model frozen, meaning their weights did not change during training. On top of this base, we added a custom classification head. This head includes a Flatten layer to convert the 2D features into a 1D vector, followed by a Dense layer with 256 neurons using the ReLU activation function. To control overfitting, We included a Dropout layer with a rate of 0.5. Finally, We used a Dense layer with a single neuron and Sigmoid activation for the final cat vs. dog prediction.

2.Training Process

The training process for this model was different because we used a larger image resolution of 224x224 pixels, which is the standard input size for VGG16. This allowed the model to see more details in the images compared to my previous 128x128 models. We used the Adam optimizer and the binary cross-entropy loss function, just like in our earlier experiments.

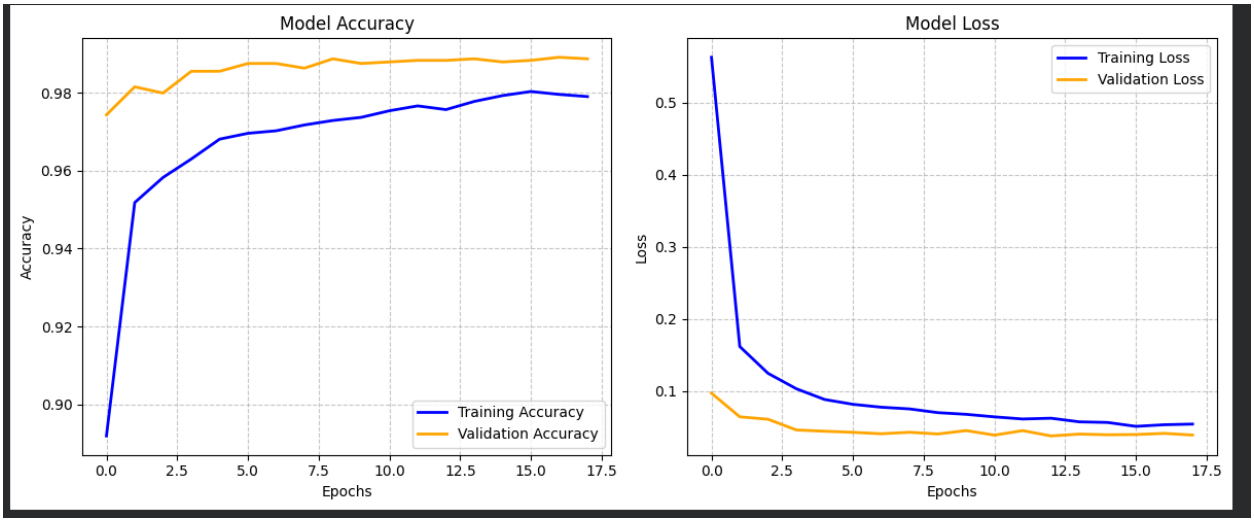
```
train_dir = 'data/cats_dogs/train'
validation_dir = 'data/cats_dogs/validation'
test_dir = 'data/cats_dogs/test'

IMG_SIZE = 224
BATCH_SIZE = 32

train_datagen = ImageDataGenerator(
    preprocessing_function=vgg16_preprocess,
    rotation_range=40,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

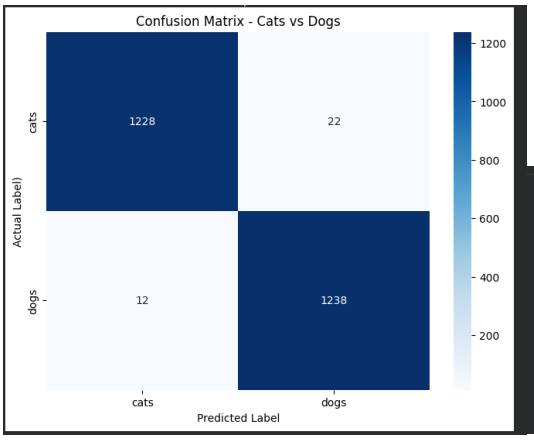
```
validation_datagen = ImageDataGenerator(
    preprocessing_function=vgg16_preprocess
)

test_datagen = ImageDataGenerator(
    preprocessing_function=vgg16_preprocess
)


print("Loading Training Data...")
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

print("Loading Validation Data...")
validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical'
)

print("Loading Test Data...")
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False  # IMPORTANT: Do tions aligned with true labels
)
```

Since the base layers were frozen, the model only had to learn the weights for the final dense layers we added. This made the training much faster and more stable. We focused on ensuring that the Dropout layer effectively prevented the model from simply memorizing the training data.

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau

checkpoint = ModelCheckpoint(
    filepath='best_cats_dogs_model.h5',
    monitor='val_accuracy',
    save_best_only=True,
    mode='max',
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=2,
    min_lr=1e-6,
    verbose=1
)


early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True,
    verbose=1
)

epochs = 20

print("Training Starting...")
history = model.fit(
    train_generator,
    epochs=epochs,
    validation_data=validation_generator,
    callbacks=[checkpoint, reduce_lr, early_stopping]
)
```

3.Experimental Results






```
Classification Report:

              precision    recall  f1-score   support

        cats       0.99      0.98      0.99      1250
        dogs       0.98      0.99      0.99      1250

    accuracy                           0.99      2500
   macro avg       0.99      0.99      0.99      2500
weighted avg       0.99      0.99      0.99      2500
```

This experiment was the most successful part of our project. The VGG16 model reached a level of accuracy that was much higher than any of our previous custom models. In the final evaluation, this model achieved a test accuracy of **98.64%** and a very low loss of **0.0466**.

```
Evaluating the model using Test Data
44/79 ━━━━━━━━━━━━━━━━━━━━ 8s 240ms/step - accuracy: 0.9864 - loss: 0.0422/
  warnings.warn(str(msg))
79/79 ━━━━━━━━━━━━━━━━━━━━ 20s 247ms/step - accuracy: 0.9854 - loss: 0.0478

Last Test Loss value: 0.0466
last Test Accuracy value: 98.64%
```

When we compared this to our previous "improved" model, which had an accuracy of 95.92%, the difference was clear. The VGG16 model was much better at generalizing and identifying the differences between cats and dogs. The training and validation curves were very smooth, showing that the model learned steadily without major overfitting issues. **This is now our final and best-performing model.**

Comparison Table:

The following table shows the clear difference between the base model and the improved model.

Metric	Implementation 1	Implementation 2 Final Model (Transfer Learning)
Test Accuracy	82.13%	98.64%
Training Accuracy	99.28%	97.98%
Overfitting Gap	17% (High)	0.466%
Stability	Low	High

Discussion

In this project, we went through a long process of trial and error to build a high-performing cat and dog classifier. This discussion highlights what we discovered during our experiments.

1.What Worked Well

The most successful part of this project was using **Transfer Learning with the VGG16 model**. It was much more powerful than any custom model we built from scratch. Because VGG16 was already trained on millions of images, it already knew how to identify shapes and textures. Additionally, **Data Augmentation** (rotating and flipping images) worked very well to stop the model from simply memorizing the training data. Using a larger image resolution of **224x224** also helped the model see more details compared to the smaller 128x128 size.

2.What Did Not Work

The biggest failure was our attempt to **retrain the original base model**. We thought that more training time would fix the errors, but it actually made the model worse. Since the base model was already overfitting, retraining it just caused it to "memorize" the noise in the training set even more. We also found that building a very deep custom model without techniques like **Batch Normalization** or **Dropout** led to very unstable training results.

3.Challenges Faced

```
479/625 ━━━━━━━━━━━━━━━━━━━━ 27s 191ms/step - accuracy: 0.7936 - loss: 0.5264/usr/local/lib/python3.12/dist-packages/PIL/TiffImagePlugin.py:950: UserWarning: Truncated File Read
  warnings.warn(str(msg))
625/625 ━━━━━━━━━━━━━━━━━━━━ 128s 197ms/step - accuracy: 0.7992 - loss: 0.5016 - val_accuracy: 0.8983 - val_loss: 0.2404
Epoch 2/30
```

We faced several challenges during the implementation. First, the dataset contained some **corrupted or empty files** which caused the code to crash. We had to write a specific script to clean the folders. Second, training deep models like VGG16 takes a lot of computing power. We had to manage our **Google Colab GPU limits** carefully. Finally, finding the right "learning rate" was difficult; if it was too high, the model wouldn't learn, and if it was too low, the training took too long.

```
from PIL import ImageFile
import warnings

ImageFile.LOAD_TRUNCATED_IMAGES = True

# Filter out the specific 'Truncated File Read' UserWarning from PIL
warnings.filterwarnings("ignore", category=UserWarning, module='PIL.TiffImagePlugin')
```

```python
from PIL import Image

dataset_path = 'PetImages'
classes = ['Cat', 'Dog']

for cls in classes:
    path = os.path.join(dataset_path, cls)
    for img_name in os.listdir(path):
        img_path = os.path.join(path, img_name)
        try:
            img = Image.open(img_path)
            img.verify()
        except (IOError, SyntaxError) as e:
            print(f'Removing (Corrupted): {img_path}')
            os.remove(img_path)
```

```
Removing (Corrupted): PetImages/Cat/666.jpg
Removing (Corrupted): PetImages/Cat/Thumbs.db
Removing (Corrupted): PetImages/Dog/11702.jpg
Removing (Corrupted): PetImages/Dog/Thumbs.db
/usr/local/lib/python3.12/dist-packages/PIL/TiffImagePlugin.py
  warnings.warn(str(msg))
```

```python
import random
import os

def balance_folder(folder_path, target_count):
    current_images = os.listdir(folder_path)
    if len(current_images) > target_count:
        to_remove = random.sample(current_images, len(current_images) - target_count)
        for img in to_remove:
            os.remove(os.path.join(folder_path, img))

# Define min_count based on previously calculated dog_count and cat_count
min_count = min(dog_count, cat_count)

balance_folder(os.path.join(dataset_path, 'Cat'), min_count)
balance_folder(os.path.join(dataset_path, 'Dog'), min_count)

print("Dataset successfully cleaned and balanced!")
```

```
Dataset successfully cleaned and balanced!
```

Check the Data Base Imbalanced


```python
import random
import os

def balance_folder(folder_path, target_count):
    current_images = os.listdir(folder_path)
    if len(current_images) > target_count:
        to_remove = random.sample(current_images, len(current_images) - target_count)
        for img in to_remove:
            os.remove(os.path.join(folder_path, img))

# Define min_count based on previously calculated dog_count and cat_count
min_count = min(dog_count, cat_count)

balance_folder(os.path.join(dataset_path, 'Cat'), min_count)
balance_folder(os.path.join(dataset_path, 'Dog'), min_count)

print("Dataset successfully cleaned and balanced!")
```

```
Dataset successfully cleaned and balanced!
```


4.Lessons Learned

This project taught us that **"more training" is not always the answer**. The quality of the model architecture and the variety of the data are much more important. We learned that Transfer Learning is a very efficient way to solve complex problems with limited time. Most importantly, we learned how to use **Early Stopping** to save time and get the best possible version of a model before it starts to overfit.

5.Summary of Outcomes

Overall, the project was a success. we started with a basic CNN model and improved it step-by-step. Our final model, based on **VGG16**, achieved a test accuracy of **98.64%**, which is an excellent result for this task. The system can now reliably distinguish between cats and dogs with very high confidence.

6.Limitations

Even though the accuracy is high, the model is quite large in file size, which might make it slow to run on a mobile phone or a small device. Also, the model was only trained on "clean" photos of cats and dogs; it might struggle with images where animals are far away or partially hidden.

7.Future Improvements

In the future, we want to try **Fine-tuning**, where we unfreeze some of the top layers of VGG16 to see if we can reach 99% accuracy. We also want to experiment with **MobileNet**, which is a smaller and faster model designed for mobile apps. Finally, We would like to create a simple web interface where users can upload their own pet photos to get an instant prediction.

8.Save the Model

Save Location: First We save the model to Drive and Then the saved model file has been directly uploaded to our project's public GitHub repository. It can be safely accessed and downloaded via this link**:https://github.com/RashmikaKDH/Cat-and-Dog-Classification-using-CNN.git**